PES University

Department of Computer Science and Engineering

UE24CS341A Software Engineering

Mini Project Report

Event Management System (EMS)

A web application for creating events, registering attendees, managing waitlists, checking in with QR tickets and reporting attendance

Aug to Nov 2026

Group 7 (to confirm)

Name	SRN	Feature owned
Varun Rao	PES1UG24CS520	F1 Authentication and role management
Thanishq Raam Patil	PES1UG24CS502	F2 Event creation and management
Vishruta	PES1UG24CS538	F3 Event registration and ticketing
Nithin Kumar M	PES1UG25CS829	F4 Check-in, notifications and reports

Faculty: _____

Repository: https://github.com/SE-MINIPROJECT-ORG/SWE-MiniProject

Contents

1. Project overview (proposal)
2. Requirements
3. Project plan
4. Architecture and design
5. Agile execution: backlog, sprints and CI/CD
6. Testing and validation
7. Deviations from the design documents
8. Screenshots of the working system
9. Conclusion
Appendix: where to find each document

1. Project overview

Problem

Events in colleges are often run with forms, spreadsheets and group messages. Seats get overbooked, confirmations get lost, waitlists are handled by hand, and organizers do not know who actually attended.

Solution

EMS puts the whole event lifecycle in one web application. Organizers create and publish events with a fixed capacity. Attendees register and get a ticket with a QR code; when an event is full they join a waitlist and are promoted automatically when a seat frees up. Organizers check attendees in by ticket ID and get an attendance report and a CSV list. Email and in-app notifications keep everyone informed, and admins manage users, roles and categories.

Users

User	What they do
Visitor	Browses and searches events, signs up
Attendee	Registers, joins waitlists, views tickets and notifications, cancels
Organizer	Creates, edits, publishes and cancels events; checks attendees in; views reports
Admin	Manages users, roles and categories; views platform totals

Features and ownership

Each member owned one feature end to end: requirements, design, code, tests and demo.

Feature	Owner	Main functions
F1 Authentication and role management	Varun Rao	Sign-up, login, lockout, logout, roles, admin user management, profile
F2 Event creation and management	Thanishq Raam Patil	Create, validate, edit, publish, cancel events; categories; search and filter
F3 Event registration and ticketing	Vishruta	Register, waitlist and promotion, cancel, my registrations, QR ticket, organizer list
F4 Check-in, notifications and reports	Nithin Kumar M	Check-in, emails, reminders, in-app notifications, report, CSV, admin dashboard

Technology: React (frontend), Node.js with Express (REST API), PostgreSQL (database), Docker Compose, GitHub Actions.

2. Requirements

The requirements are specified in the SRS (IEEE format, `docs/submission/SRS_EMS_v2.0.docx`). Every requirement has an ID, a priority, measurable acceptance criteria and a linked test case.

Type	Count	ID format	Example
Functional	30	EVT-F-###	EVT-F-022: an attendee who registers for a full event is put on a first-in-first-out waitlist

Type	Count	ID format	Example
Non-functional	6	EVT-NF-###	EVT-NF-001: 95% of API requests complete within 2 s with 100 concurrent users
Security objectives	3	SO-#	SO-1 Confidentiality, SO-2 Integrity, SO-3 Accountability and Availability
Security requirements	7	EVT-SR-###	EVT-SR-001: passwords stored only as salted bcrypt hashes

Non-functional requirements

ID	Requirement	Target
EVT-NF-001	Performance	95% of requests within 2 s at 100 concurrent users
EVT-NF-002	Availability	99% monthly uptime
EVT-NF-003	Usability	Register within 3 clicks; layouts from 360 px to 1920 px
EVT-NF-004	Correctness under load	No overbooking under 500 simultaneous registrations
EVT-NF-005	Maintainability	At least 70% unit test coverage, quality gate in CI
EVT-NF-006	Compatibility	Latest two versions of Chrome, Firefox and Edge

Security requirements

ID	Requirement
EVT-SR-001	Passwords stored only as salted bcrypt hashes, never logged
EVT-SR-002	Signed session tokens (JWT) that expire after 60 minutes
EVT-SR-003	All input validated on the server; parameterized SQL queries
EVT-SR-004	HTTPS with TLS 1.2 or higher; plain HTTP redirected
EVT-SR-005	Ownership checks: users only see and change their own data
EVT-SR-006	Audit log of logins, role changes, event cancellations and check-ins
EVT-SR-007	Rate limiting: 100 requests and 10 logins per minute per IP

Use case diagrams

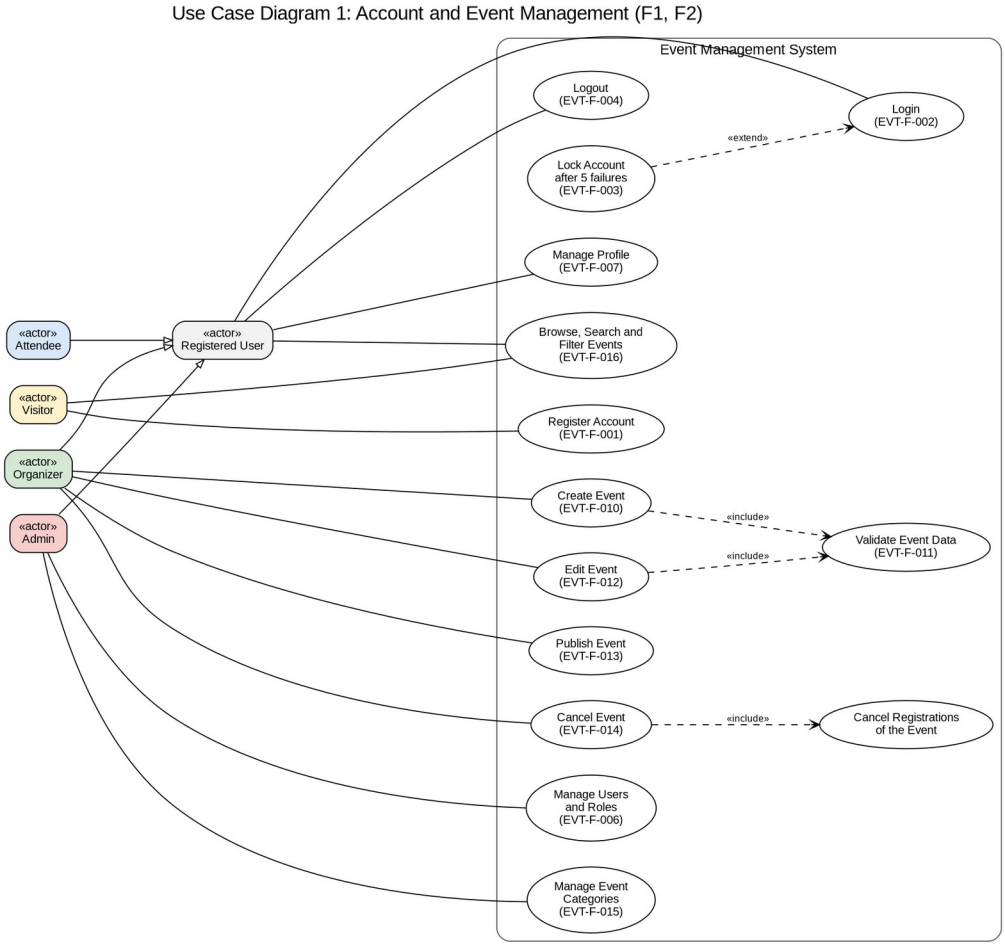


Figure 2.1: Account and event management (F1, F2)

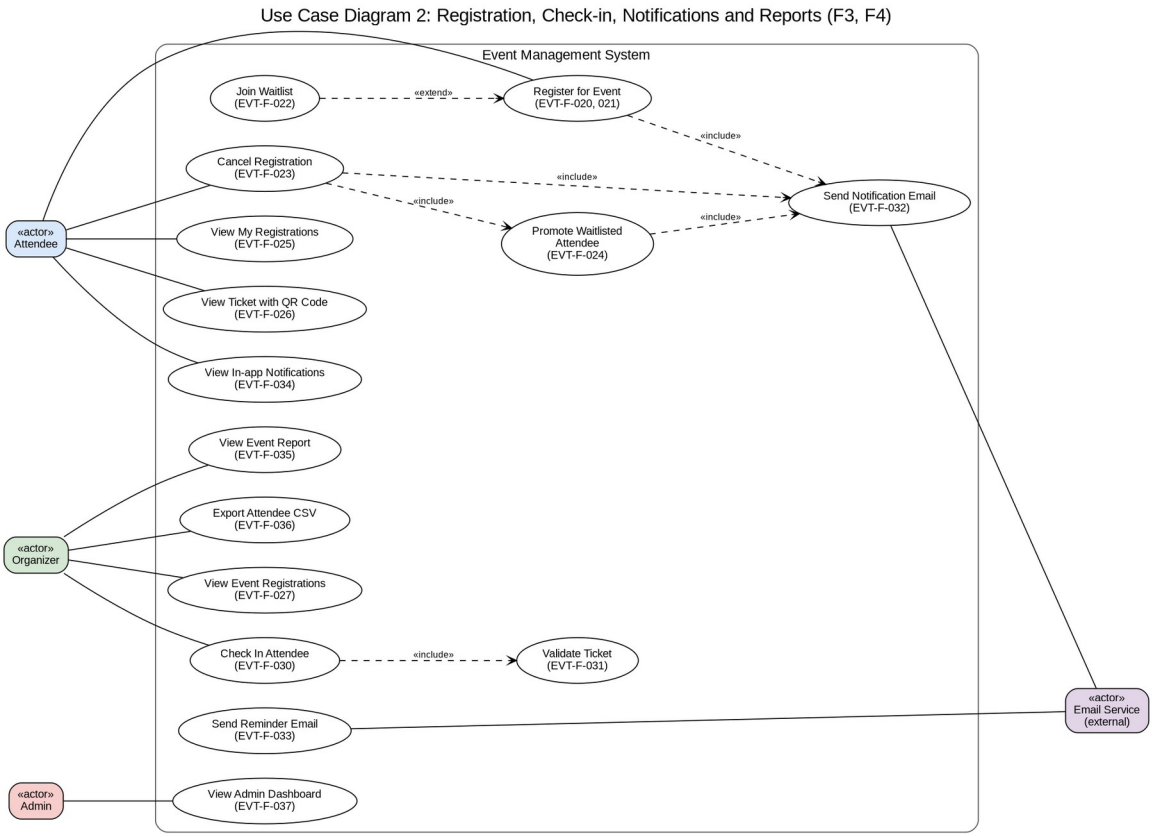


Figure 2.2: Registration, check-in and reports (F3, F4)

3. Project plan

The full plan is in `docs/project-plan/project-plan.pdf`.

Lifecycle model: Agile (Scrum)

- **Why:** the course requires Agile; each member owns one feature, which fits per-member user stories; working software is demonstrated at the end of each sprint; the teacher reviews between sprints, and changes are recorded at each sprint review.
- **How:** an up-front phase ("Sprint 0") for the IEEE documents and project setup, then two one-week sprints, each ending with a working product, a video and a sprint review.

Work breakdown structure

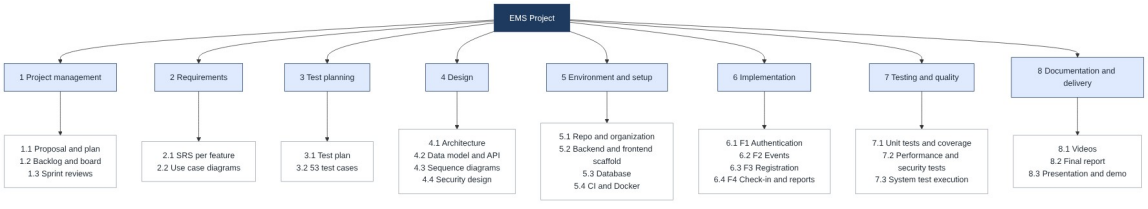


Figure 3.1: Work breakdown structure (8 work packages)

Effort estimation

Method	Result
Story points	167 points in total (Sprint 1: 86, Sprint 2: 81); per member 41 to 43 points
Bottom-up estimate	425 person-hours = **2.7 person-months** (implementation at about 1.25 hours per point)
COCOMO (basic, organic, 4.6 KLOC)	11.9 person-months, 6.4 months, used as an upper bound: COCOMO assumes industrial overheads and code written from scratch, while this project reuses frameworks and libraries

Schedule

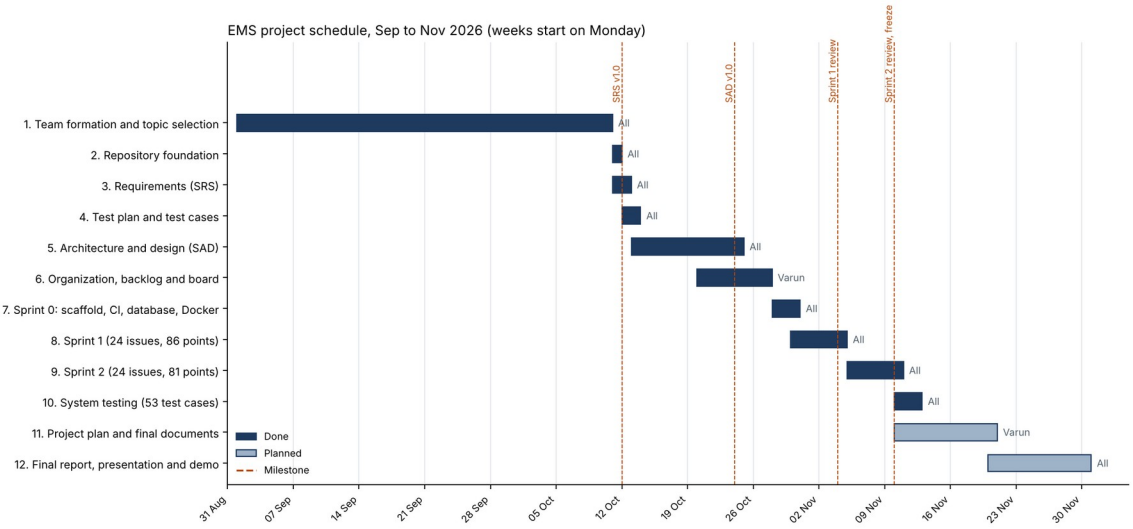


Figure 3.2: Gantt chart built from the actual dates in the repository history

4. Architecture and design

The full design is in the SAD (IEEE format, `docs/submission/SAD_EMS_v2.0.docx`).

Architecture pattern: layered monolith

One deployable backend split into four layers: **Presentation** (React app), **API** (Express routes, security middleware), **Service** (business rules, one module per feature) and **Data** (repositories with SQL).

- **Why:** clear separation of concerns; each member works in their own module; seat allocation and waitlist promotion need one database transaction, which is simple inside one service; easy to run on a laptop with Docker.
- **Rejected:** microservices (network calls and distributed transactions for a four-person, one-semester project) and serverless functions (cold starts and scattered state make seat allocation and scheduled reminders harder).

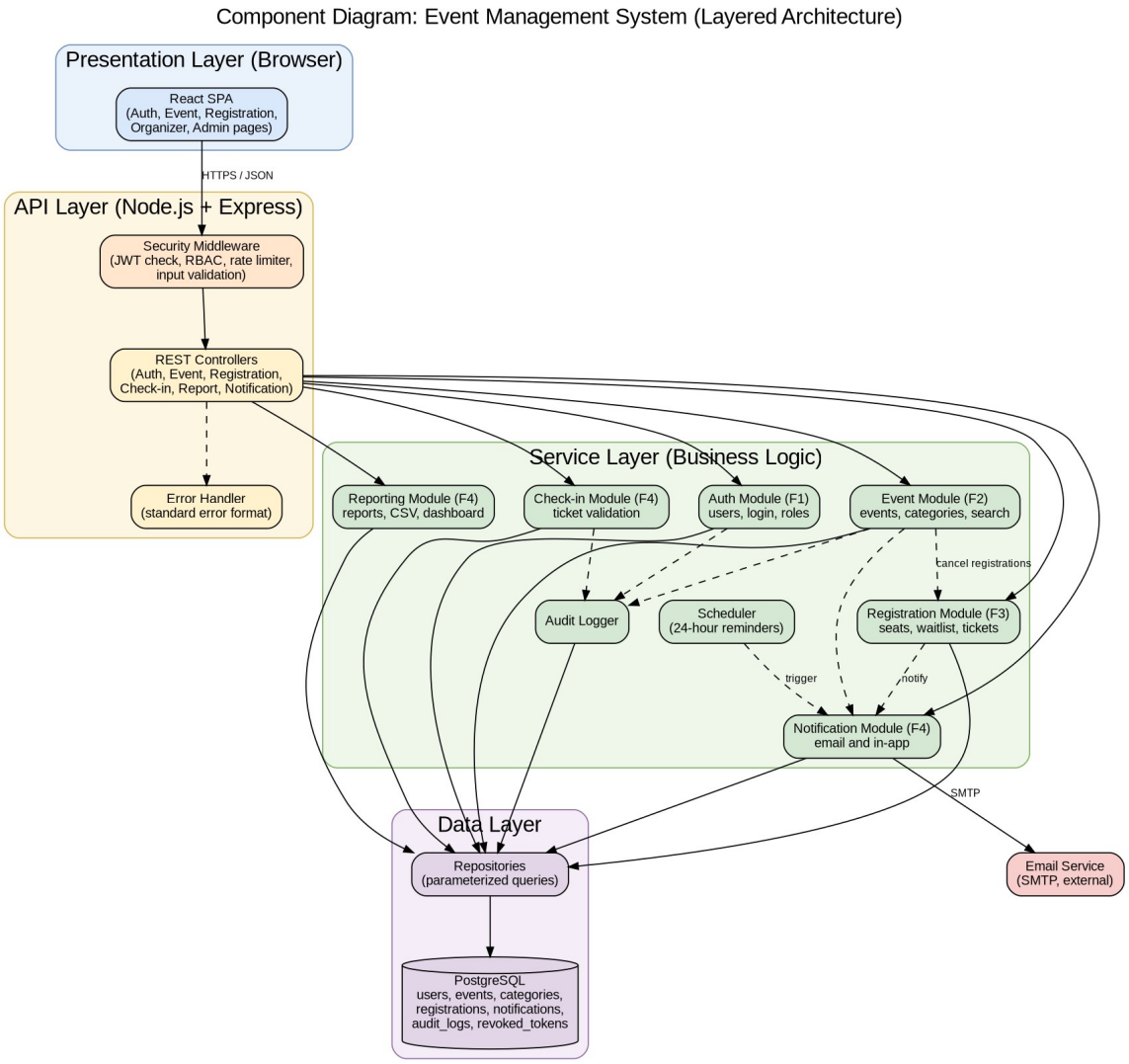


Figure 4.1: Component diagram

Data model

Table	Purpose
users	Accounts, roles, active flag, failed-login counter and lock time
categories	Event categories
events	Event details, capacity and status (Draft, Published, Cancelled)
registrations	Status (Confirmed, Waitlisted, Cancelled, Checked in), ticket ID, queue position, reminder flag
notifications	In-app notifications per user
audit_logs	Security events (who, what, when)
revoked_tokens	Logged-out tokens, kept until they expire

Key design decisions

Decision	Reason	Requirement
Registration and cancellation run in one	Two people can never take the last seat at the	EVT-NF-004

Decision	Reason	Requirement
transaction with the event row locked	same time	
Emails are sent in the background with 3 retries	A slow mail server never fails or slows the user's request	EVT-F-032
Reminder job runs every minute and records `reminder_sent_at`	Each attendee gets exactly one reminder	EVT-F-033
Logout stores the token ID in `revoked_tokens`	A stolen token stops working after logout	EVT-F-004
One standard error format `{ error: { code, message, details } }`	The frontend shows clear messages; no stack traces leak	SAD 4.4

Sequence diagrams

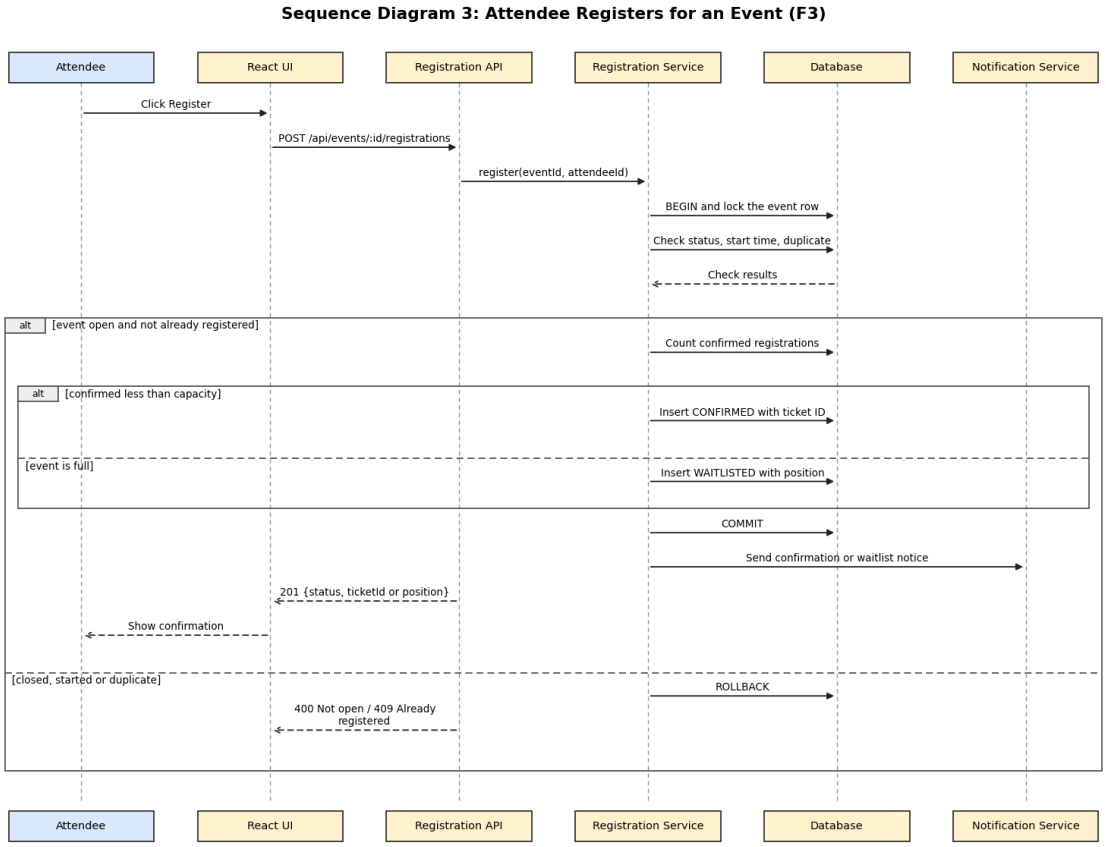


Figure 4.2: Attendee registers for an event (seat lock and waitlist)

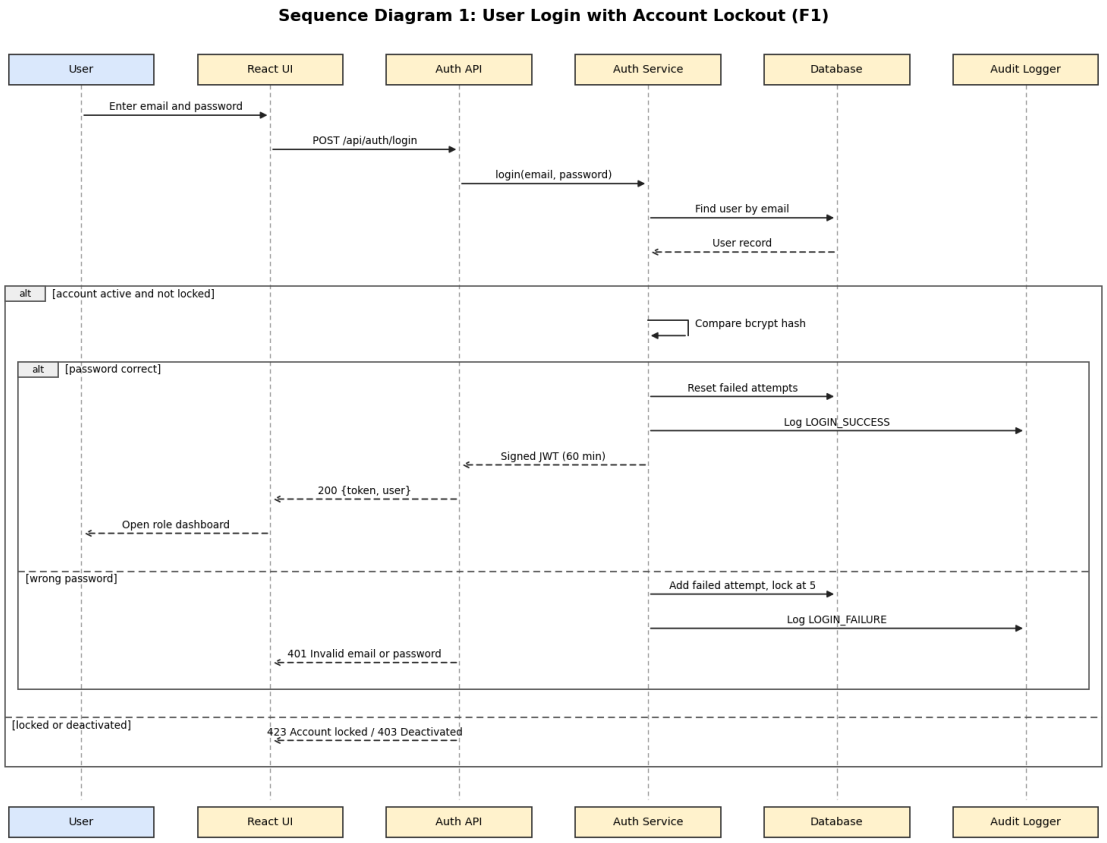


Figure 4.3: Login with account lockout. Two more diagrams (event cancellation, check-in) are in the SAD.

API design

32 REST endpoints over HTTPS with JSON, grouped by module. Examples:

Endpoint	Who	Purpose
POST /api/auth/login	Anyone	Log in, returns a 60-minute token
POST /api/events	Organizer	Create an event (saved as Draft)
POST /api/events/:id/registrations	Attendee	Register, or join the waitlist when full
POST /api/events/:id/checkins	Organizer	Check in a ticket
GET /api/events/:id/report	Organizer	Attendance report

Security architecture (STRIDE)

Threat	Example	Control
Spoofing	Guessing passwords	bcrypt, account lockout, login rate limit
Tampering	Changing a token or SQL injection	Signed JWT, parameterized queries, validation
Repudiation	Denying a role change	Audit log
Information disclosure	Reading another user's ticket	Ownership checks, HTTPS
Denial of service	Flooding the API	Rate limiting
Elevation of privilege	Attendee calling an admin endpoint	Role-based access control (HTTP 403)

5. Agile execution: backlog, sprints and CI/CD

Backlog and board

- **48 GitHub Issues**: every functional, non-functional and security requirement of the SRS plus 5 setup items, each with story points (`sp-N` labels), a priority, a feature label and an assignee.
- **GitHub Project "EMS Product Backlog"** linked to the repository, with **Sprint 1** and **Sprint 2** as milestones.

Member	Sprint 1	Sprint 2	Total points
Varun Rao	25	16	41
Thanishq Raam Patil	21	21	42
Vishruta	20	21	41
Nithin Kumar M	20	23	43
Total	**86**	**81**	**167**

Sprints

Sprint	Goal	Result
Sprint 0	IEEE documents, repository, scaffold, database, CI, Docker	SRS, Test Plan, SAD; working build pipeline
Sprint 1 (24 issues)	Accounts, events, registration with waitlist, emails, audit log, rate limiting	All 24 closed; 1-minute video; sprint review with 7 deviations
Sprint 2 (24 issues)	Remaining features, security and performance requirements, tests	All 24 closed; 2-minute video; sprint review with 8 deviations; development frozen

Development workflow

1. Each issue is developed on its own branch (`feature/<member>-<topic>`).
2. The **assigned member raises the pull request**, with `Closes #n` linking it to the backlog item.
3. **GitHub Actions** runs lint, unit tests with the coverage gate, and the frontend build.
4. A teammate reviews and approves (fixed review rotation); the repository owner merges with Squash and merge.
5. **Branch protection** on `main` blocks merging without a green CI run and one approval, and blocks direct pushes.

Sprints were ordered so that features depending on others came later (for example, notifications before waitlist promotion, and promotion before event editing). Blockers met during the sprints, and how they were resolved: a CI workflow lost in a revert (re-added in a new pull request), Docker memory limits on one laptop (WSL memory settings), and GitHub Actions queue delays (re-running the check).

6. Testing and validation

Strategy

Level	Tool	Scope
Unit	Jest and Supertest (backend), Vitest (frontend)	241 tests, run in CI on every pull request
Integration	API calls against the backend and PostgreSQL	Test cases marked (I)
System	Full system in the browser over HTTPS	Test cases marked (S)
Performance and concurrency	Team scripts (`backend/scripts`)	Load, overbooking and uptime tests
Security	Test cases TC-SEC-01 to 07	Hashing, tokens, injection, TLS, ownership, audit, rate limits

Results

Measure	Result
System test cases (53)	50 pass, 2 pass with a note, 1 partially executed (browsers: Chrome only), **0 failed**
Unit tests	205 backend + 36 frontend, all pass
Backend coverage	**86.79% lines, 78.64% branches** (target 70% lines)
Performance	95th percentile 138 ms with 100 users for 5 minutes (target 2 s)
No overbooking	3 runs of 500 simultaneous requests for 100 seats: exactly 100 confirmed each time
HTTPS	TLS 1.0 and 1.1 refused; 1.2 and 1.3 accepted; HTTP redirected

Every requirement is traced to its design, code module and test cases in the RTM (SRS section 8). Test cases changed by the sprints were updated and marked. Reports: `docs/test-reports` (unit test report, code and branch coverage report, system validation report, and one report per non-functional and security requirement).

7. Deviations from the design documents

Deviations were recorded at each sprint review and are explained in full in SAD section 4.6.1.

#	Change	Reason
1	`bcryptjs` instead of `bcrypt`	Installs on every laptop without build tools; same hash format
2	Event category is required	Needed for filtering
3	Audit log user can be empty	Failed logins for unknown emails must still be logged
4	Extra email when joining the waitlist	Attendee learns their position at once
5	Waitlisted registrations also get a ticket ID	Database column is required; the ticket is unusable until confirmed
6	Rate limits configurable	Needed for load tests; defaults match the SRS
7	Only upcoming published events are listed	Past events cannot be registered for
8	Login token kept in browser storage, not memory	Users stayed logged in after refresh; risk limited by expiry, logout revocation and CSP
9	Self-signed HTTPS certificate	Runs locally without a public domain
10	Admins can also do organizer actions	Lets an admin step in for an organizer
11	Reminder sent to late registrations too	Every attendee gets one reminder

#	Change	Reason
12	Report "Confirmed" includes checked-in attendees	Gives the expected attendance rate
13	Admin cannot change or deactivate themselves	Never leaves the system without an admin
14	Events cannot be edited or cancelled after they start	Keeps check-in and reports consistent
15	Coverage gate covers backend application code	Test scripts are tools, not application code

Two planned checks were changed in testing: the SonarQube quality gate was replaced by ESLint and the coverage gate in CI, and browser testing was done in Chrome only.

8. Screenshots of the working system

Taken from the final build running with Docker Compose over HTTPS, with sample data.

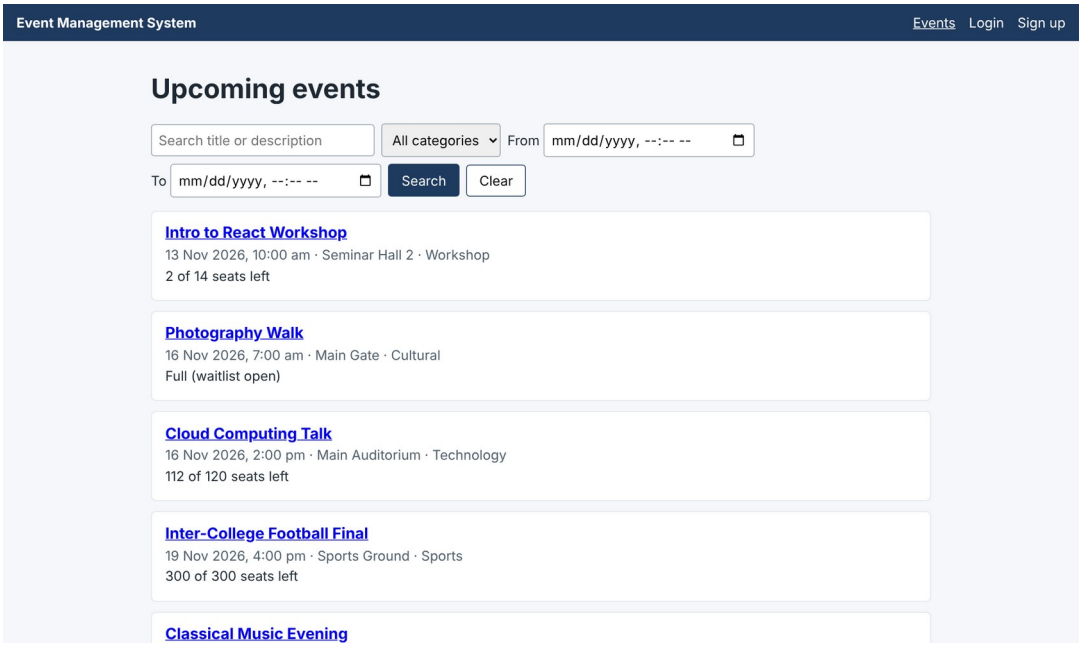


Figure 8.1: Visitor: upcoming events with search and filters (EVT-F-016)

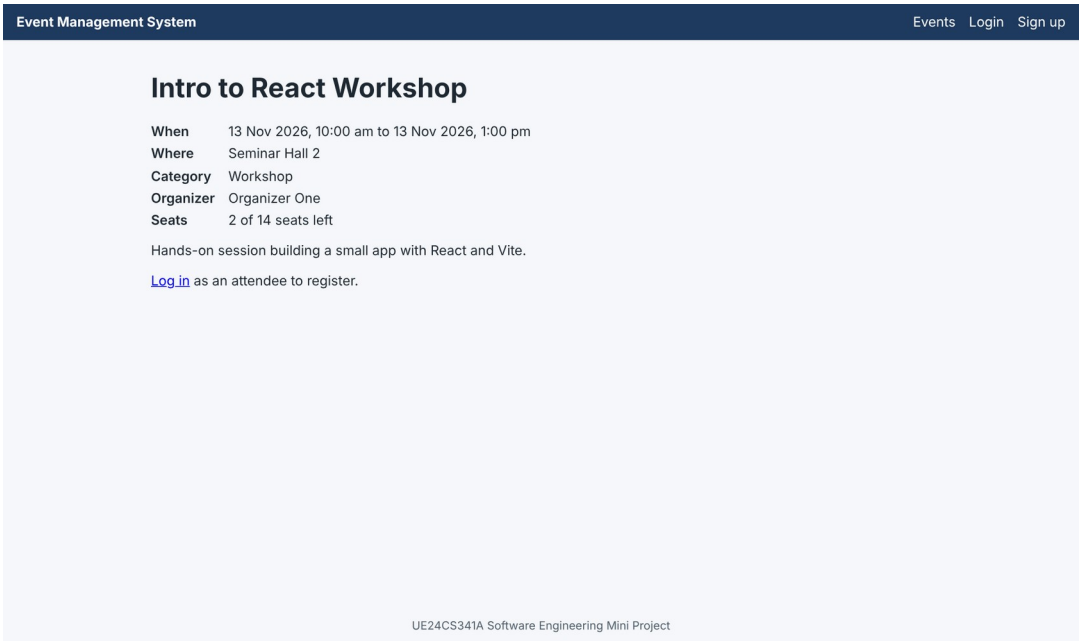


Figure 8.2: Event page with seats left

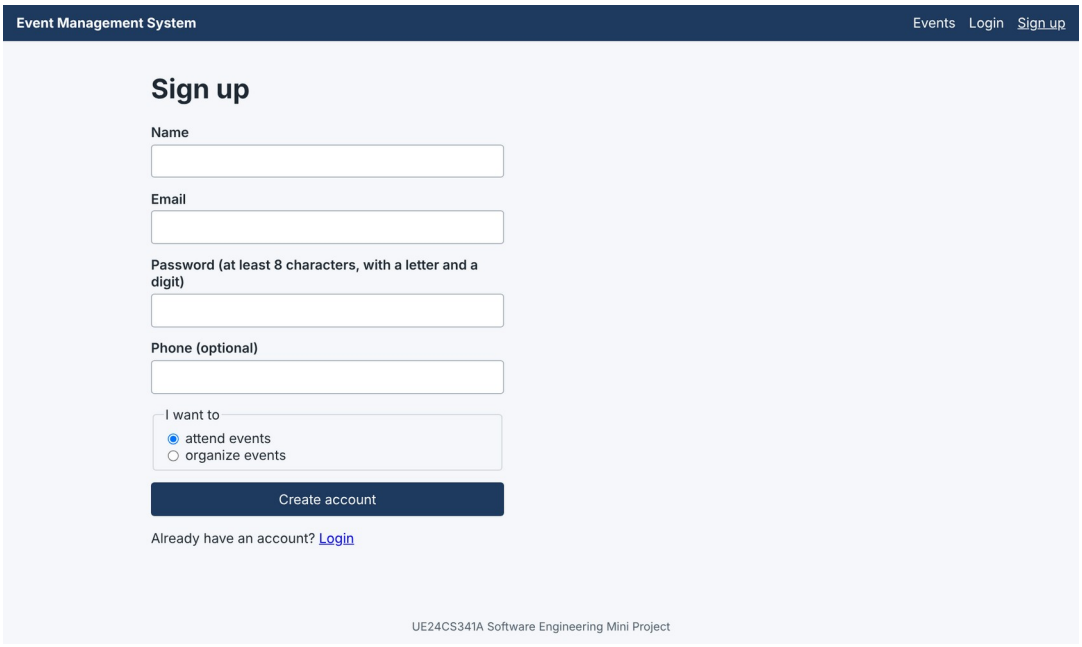


Figure 8.3: Sign-up form (EVT-F-001)

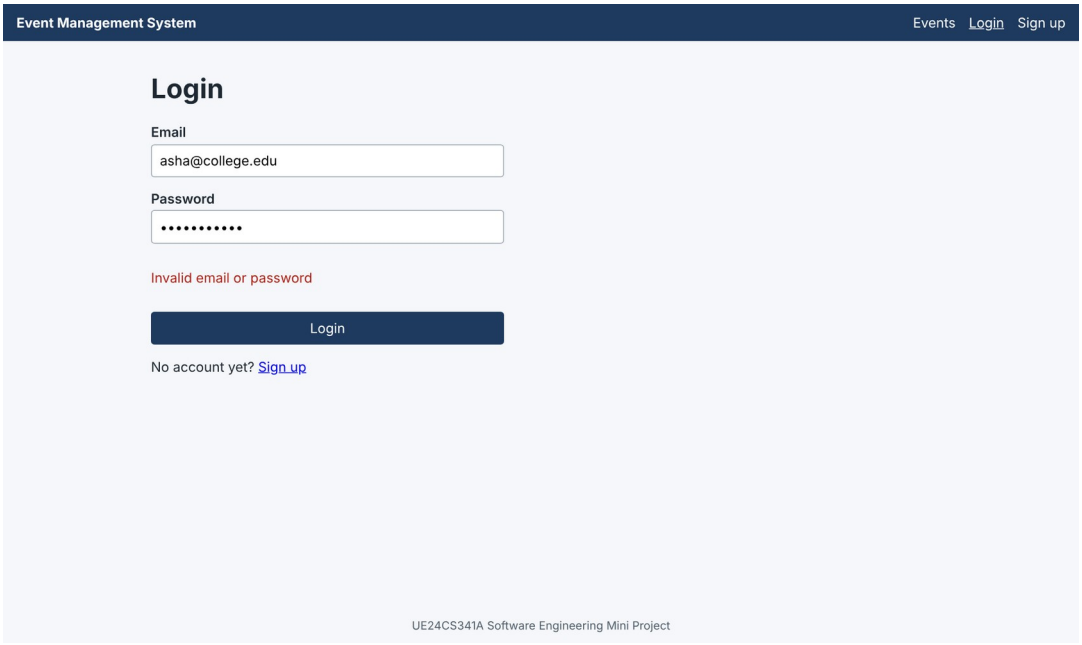


Figure 8.4: Wrong password: generic message, no hint which field is wrong (EVT-F-002)

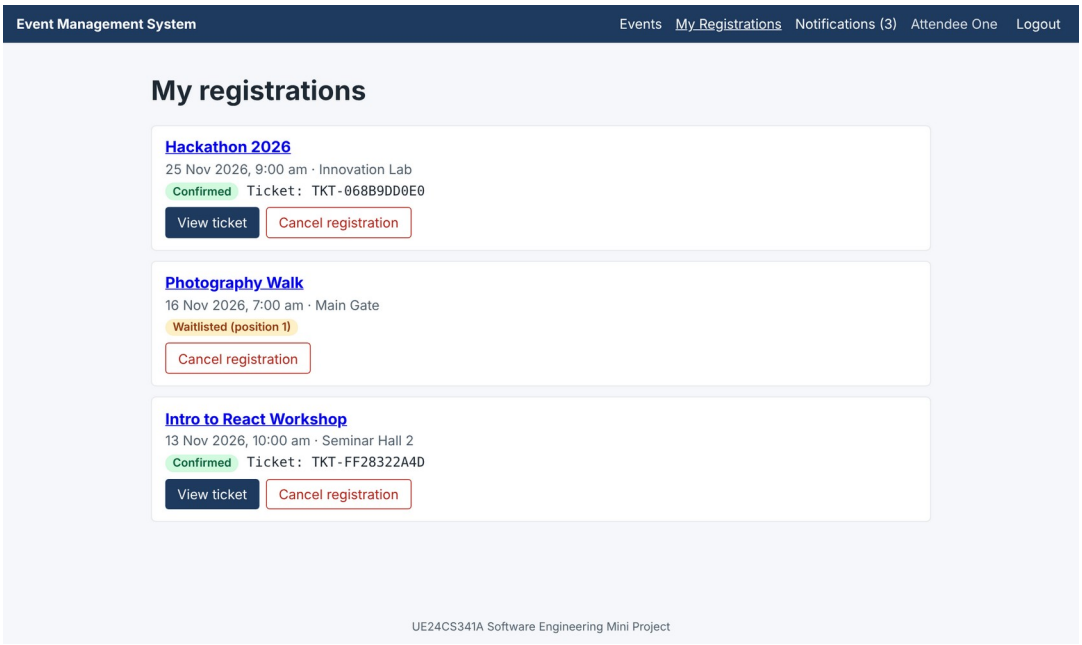


Figure 8.5: Attendee: My registrations with Confirmed and Waitlisted status (EVT-F-025)

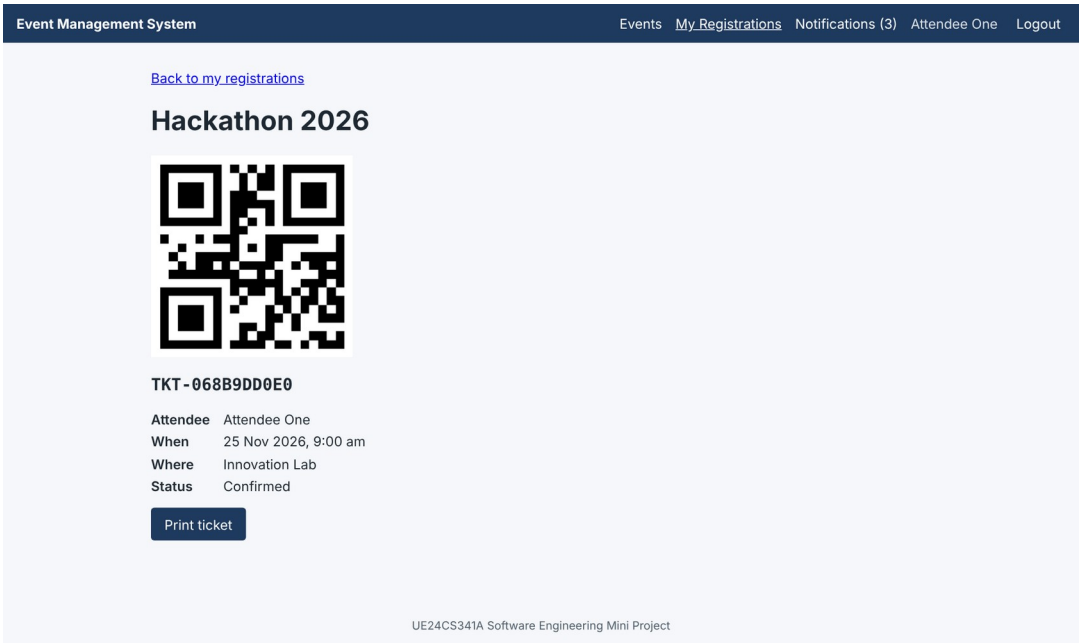


Figure 8.6: Ticket page with QR code (EVT-F-026)

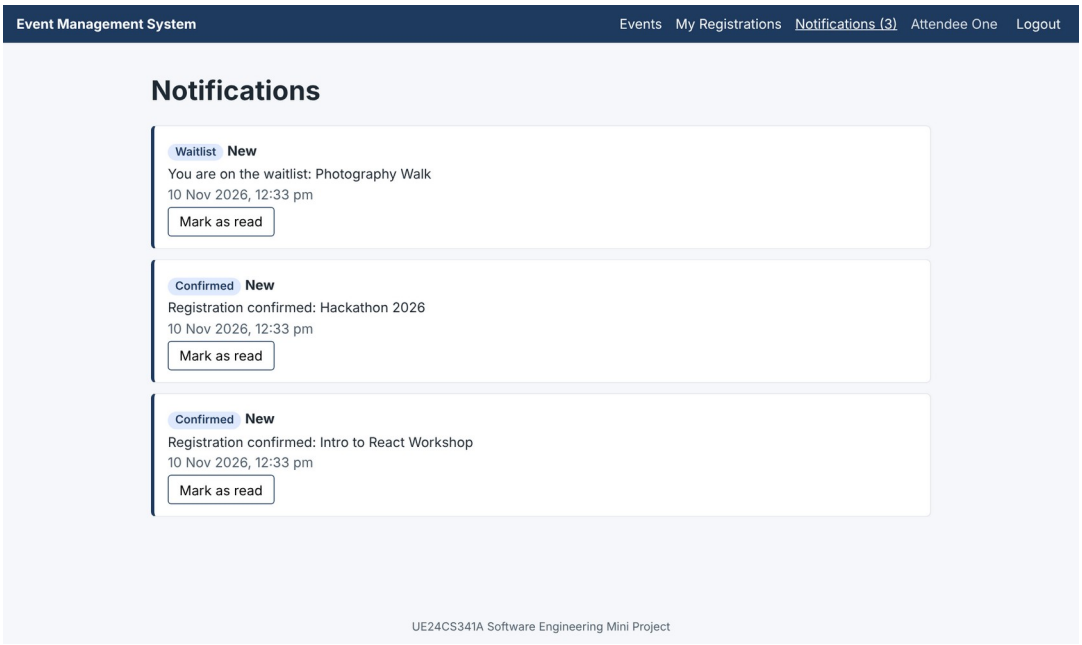


Figure 8.7: In-app notifications with unread count (EVT-F-034)

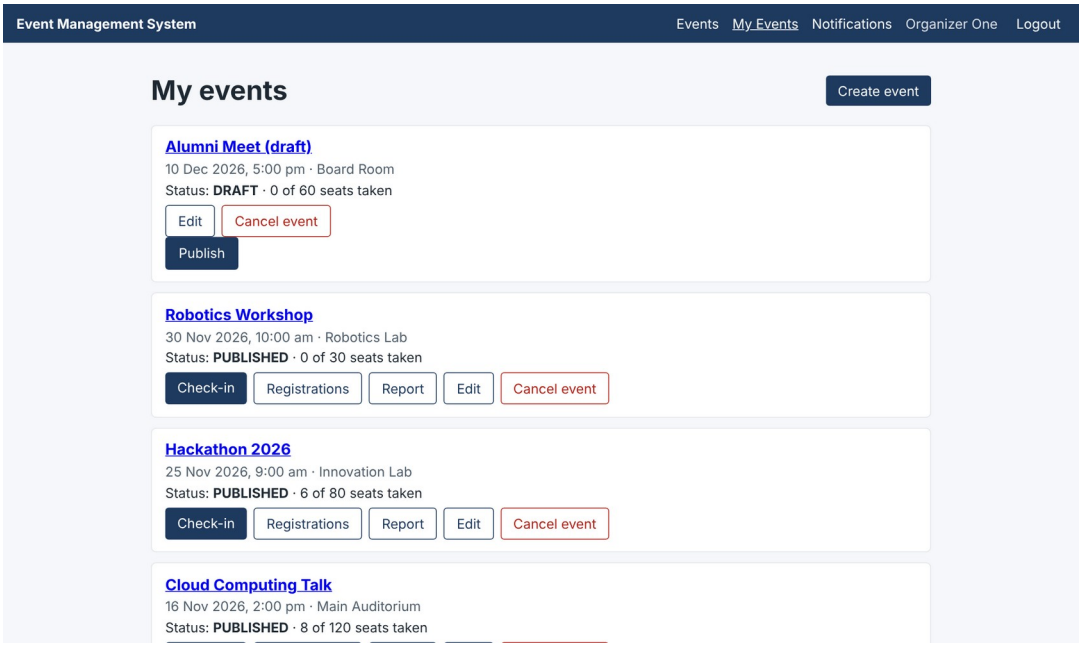


Figure 8.8: Organizer: My events with check-in, registrations, report, edit and cancel

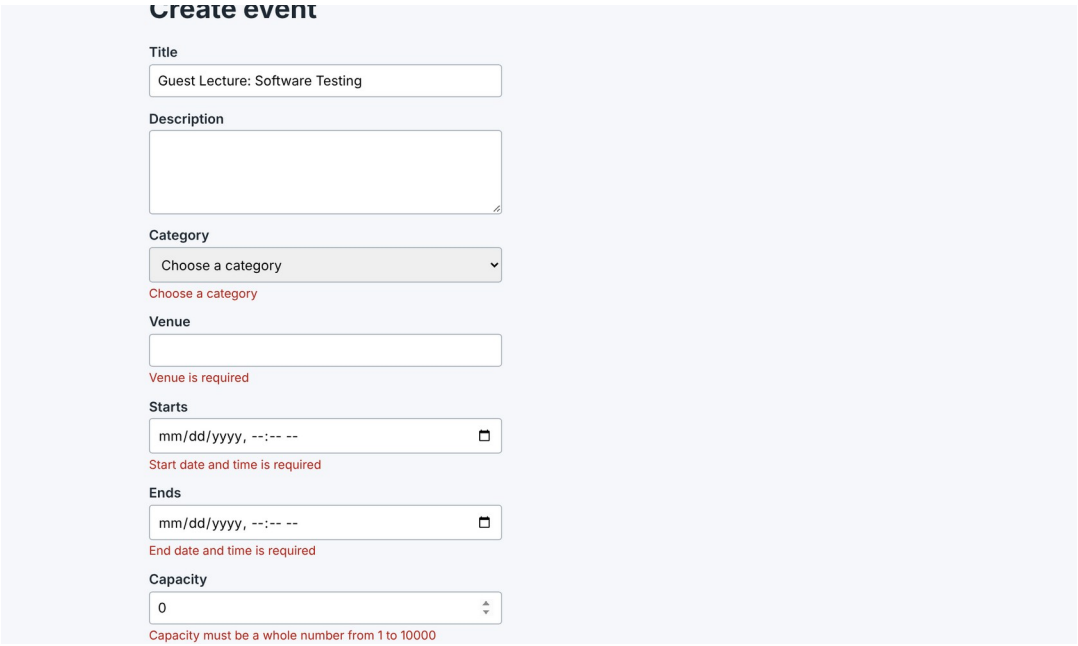


Figure 8.9: Create event: validation messages (EVT-F-011)

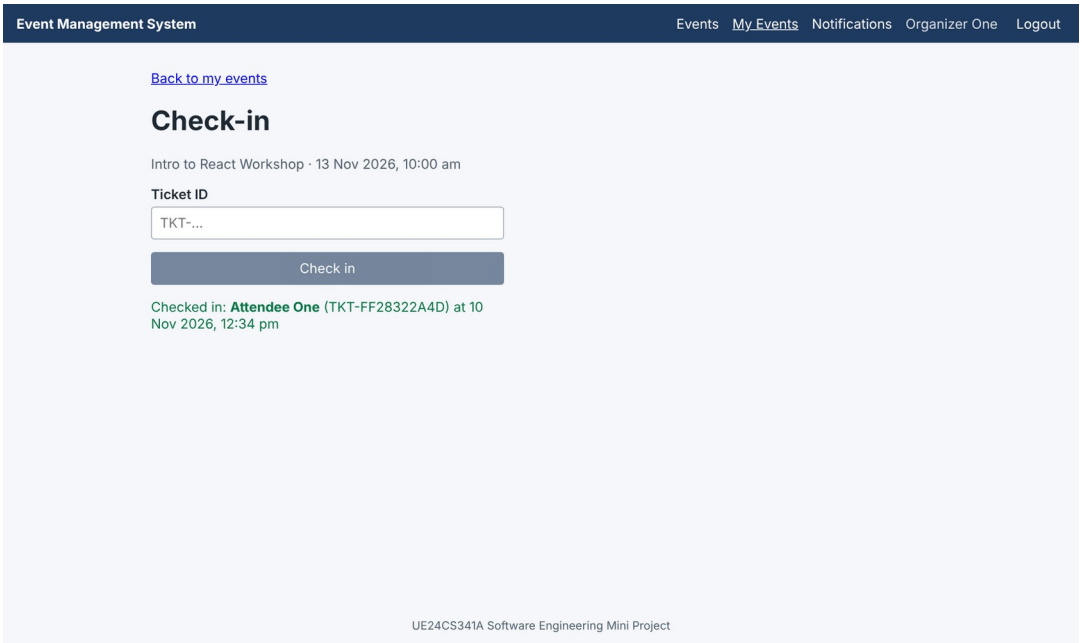


Figure 8.10: Check-in by ticket ID (EVT-F-030)

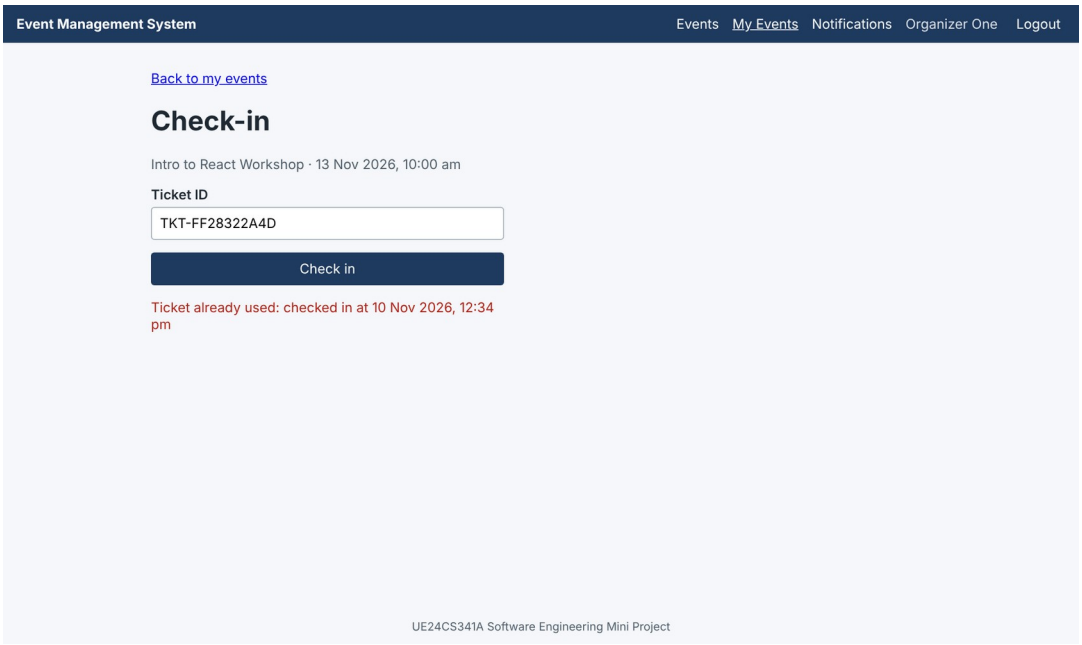


Figure 8.11: Second check-in of the same ticket is refused (EVT-F-031)

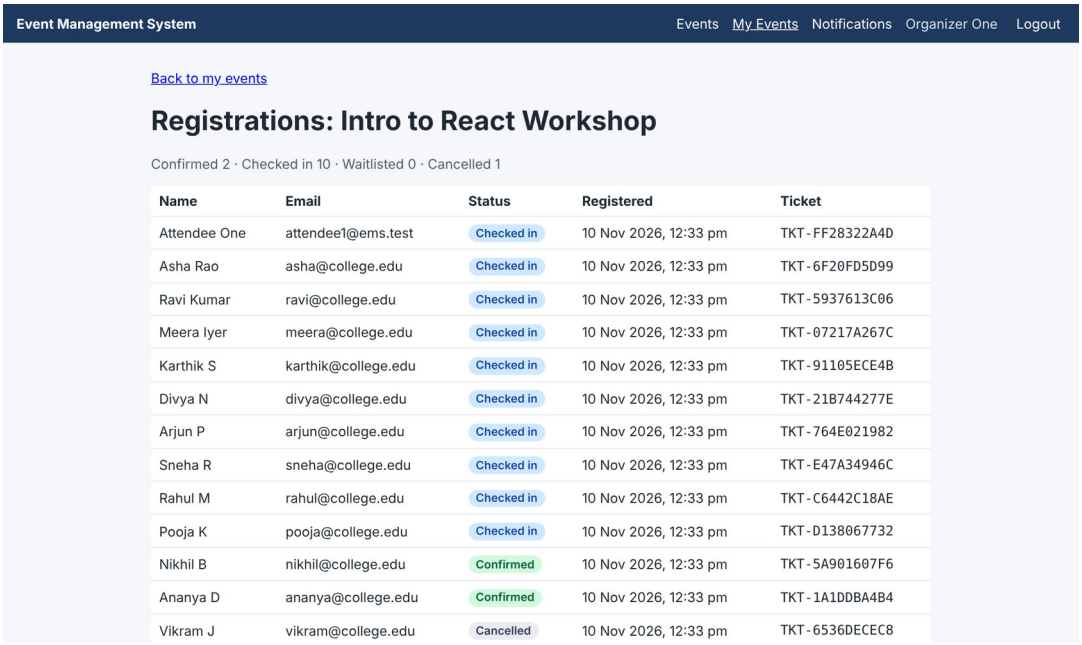


Figure 8.12: Organizer: registration list (EVT-F-027)

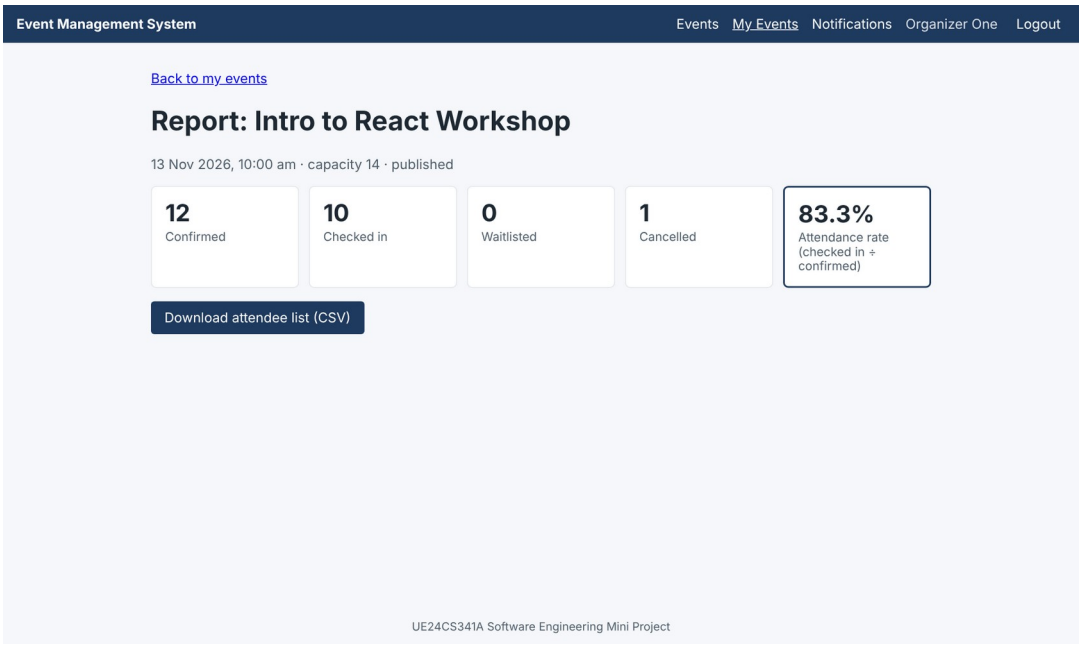


Figure 8.13: Attendance report with CSV export (EVT-F-035, EVT-F-036)

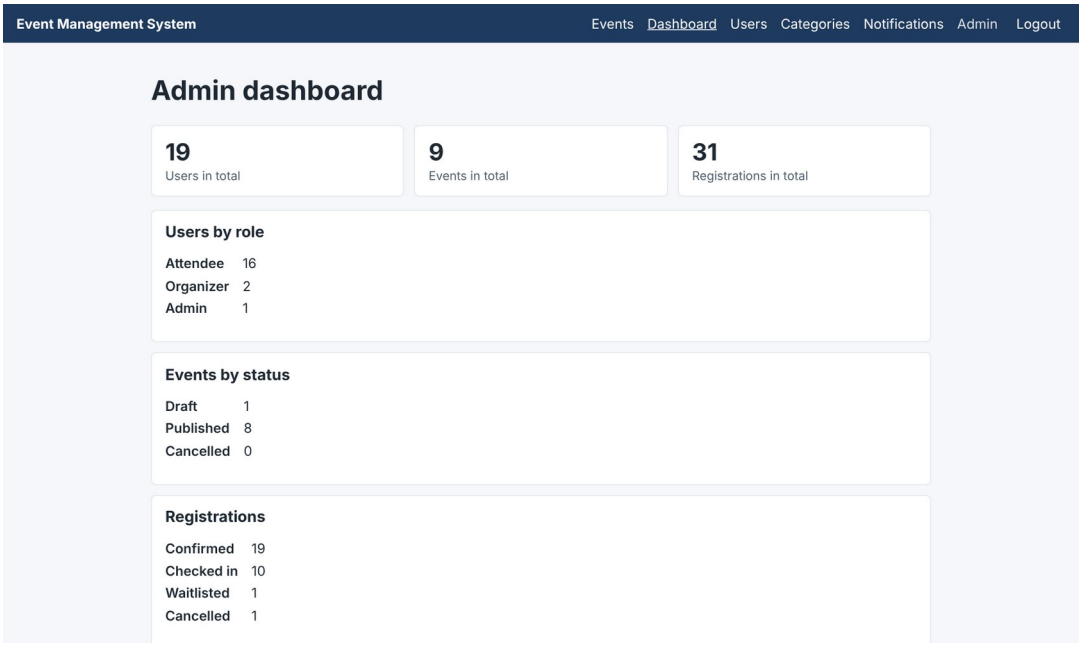


Figure 8.14: Admin dashboard (EVT-F-037)

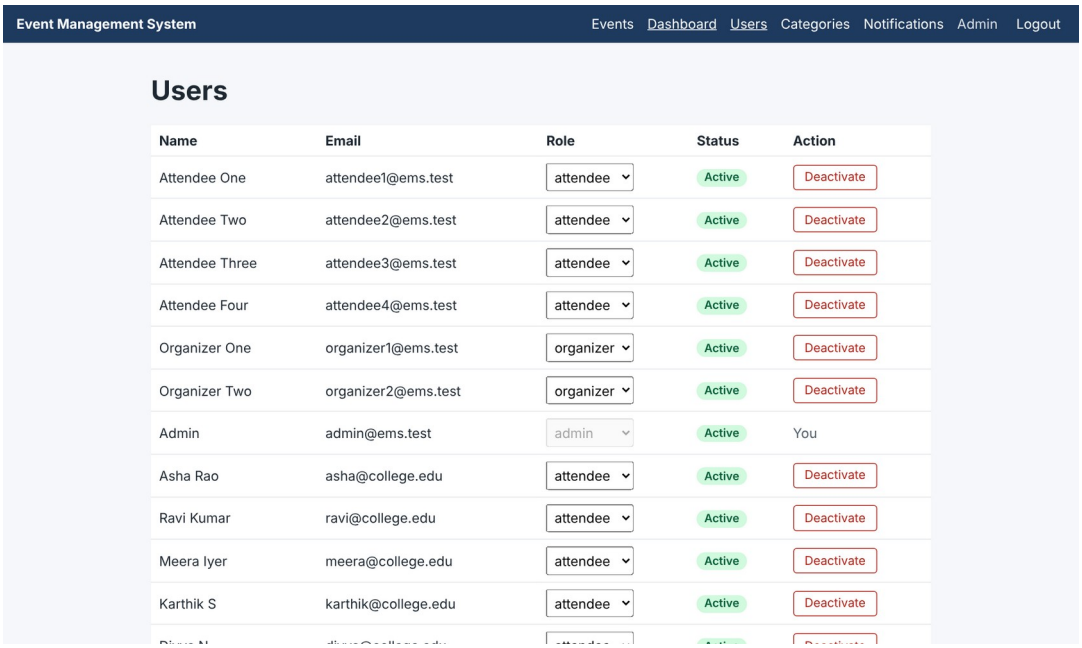


Figure 8.15: Admin: user and role management (EVT-F-006)

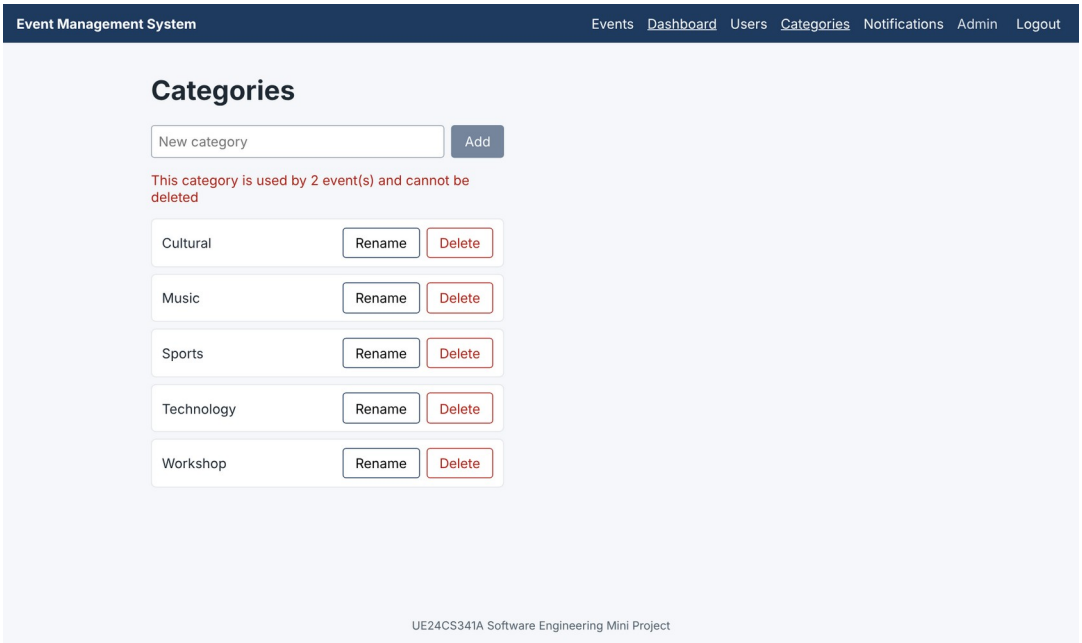


Figure 8.16: Admin: a category in use cannot be deleted (EVT-F-015)

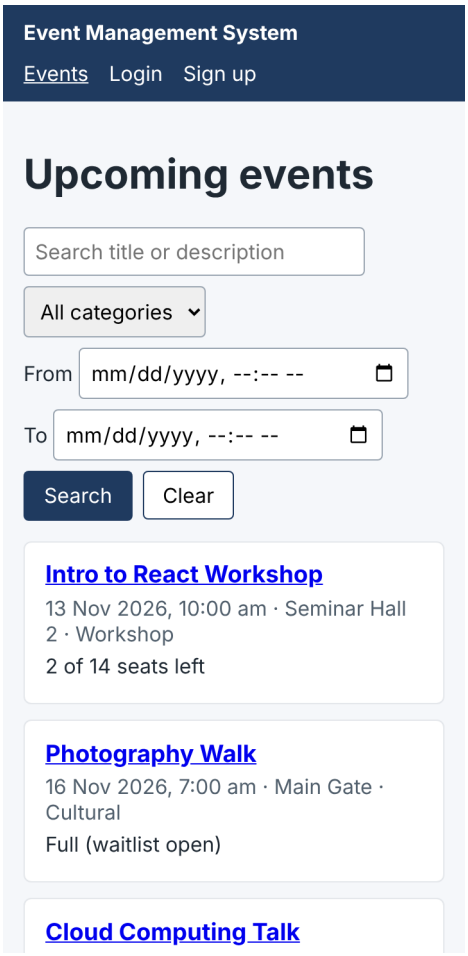


Figure 8.17: Mobile layout at 360 px (EVT-NF-003)

9. Conclusion

EMS was delivered as planned: all 48 backlog items over two sprints, by a team of four with one feature each. The system meets its functional and security requirements and its measurable non-functional targets, with no failed test cases.

What worked

- **Requirements first, with IDs and acceptance criteria:** the same IDs carried through the backlog, the pull requests, the tests and the traceability matrix, so progress and coverage were always visible.
- **Feature ownership:** each member could design, build, test and explain one complete feature.
- **CI and branch protection:** every change was linted, tested and reviewed before reaching `main`.
- **A fixed order for dependent work** kept the shared files free of conflicts.

Limitations and future work

- Availability was measured over a short period; a month-long measurement needs a deployed server.
- Browser testing was done in Chrome only.
- Future features (out of scope): online payments, seat maps, calendar integration, mobile apps.

Appendix: where to find each document

Document	Location in the repository
Proposal	docs/proposal/proposal.pdf
SRS, Test Plan, SAD (final, IEEE format)	docs/submission/ (Word and PDF)
Project plan	docs/project-plan/project-plan.pdf
Diagrams (sources and images)	docs/diagrams/
Sprint reviews and deviations	docs/sprint-reviews/
Sprint videos	docs/videos/sprint1-demo.mp4, sprint2-demo.mp4
Test reports	docs/test-reports/
Backlog and sprint board	GitHub Project "EMS Product Backlog"